OAuth App Setup Guides

Manthan Jha · Composio APM take-home · October 2026

Live: composio-oauth-guides-one.vercel.app (live generation on, up to 10 guides a day)
Code: github.com/manthanjha7/composio-oauth-guides
Recorded run: composio-oauth-guides-one.vercel.app/generate?replay=asana
Shorter version on my portfolio: manthanjha.vercel.app/case-studies/oauth-guides

I built a library of setup guides and an agent that writes new ones on request. For any app, a guide tells a developer whether they need their own OAuth app, how hard approval will be for what they're building, how long they'll wait on the provider, and which steps an agent can do for them.

The library has 11 guides. Type any other app into the search box and the agent researches it live in about two minutes, then saves a guide with its own link. Every fact in a guide carries a quote from the provider's own docs, and code checks that the quote is really on that page. In the last two rounds of my 18-app test, the agent got 18 and then 17 apps fully right.

If you have five minutes

1. Open the Gmail guide on the live site and switch between the four setups. Difficulty, your time and Google's wait change with each one.
2. Open "Sources" next to any line. You'll see the exact quote and the page it came from. The Verification page explains how quotes are checked.
3. Search an app that isn't in the library, such as Trello or Linear, and watch the agent work. If the daily cap has been reached, the recorded run shows the same thing.
4. In the code, the agent lives in `src/lib/agent/`, the evidence checks in `src/lib/evidence.ts`, and every test round is written up in `docs/agent/EVAL.md`.

The problem

A developer building an agent on Composio wants their users to connect Gmail, Slack or a CRM. For a prototype that's easy, because Composio lends its own approved OAuth apps. Launch is where it changes. Their own name on the consent screen, their own rate limits, extra permissions, or any of the 219 Composio toolkits without a shared app all mean registering their own OAuth app with the provider and getting it approved.

Each provider runs approval its own way, the rules are spread over many pages, and the answer depends on what you're building. Gmail shows this well:

If you're building	What Google asks for
An internal tool for your own company	No review
A public app that only sends email	App verification, usually a few days
A public agent that reads inboxes on your servers	Verification, a third-party security assessment, and a re-check every year

So I decided early that difficulty belongs to a setup, and that one rating per app would mislead people.

This matters to Composio because it grows when agents run in production. A developer waiting weeks on a provider review doesn't get there, and today nobody tells them upfront whether to expect an hour or ten weeks.

What I learned from two engineers

I had never registered an OAuth app that other people's accounts connect to, so before designing anything I asked two engineers to walk me through it.

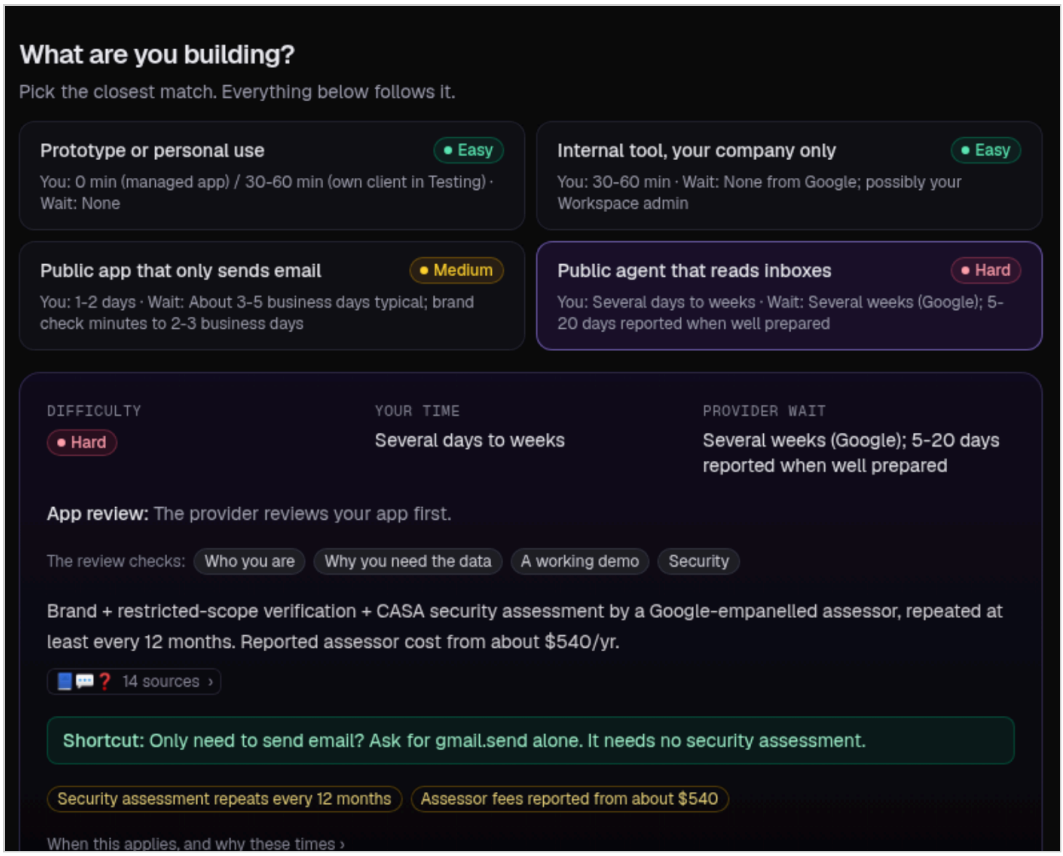
Samir is a backend engineer in his pre-final year at IIT Roorkee. He described the routine from the developer's side. For every tool, you go to that provider's developer site, create an app, copy the client ID and secret, and paste them into Composio. With several tools you do this again and again, each time in a different console. That convinced me the agent should do every step it can through an API and leave the developer only what needs a person, like logging in, accepting terms and clicking approve. It's why every step in a guide is marked as yours or the agent's.

Ayush is in his final year at IIT Roorkee and a research intern at Adobe. He took me through the whole process: what has to exist before you start (a developer account, the app registration, a redirect address, the permissions you'll ask for), the order of the steps, and the places where it tends to get stuck. What stayed with me was that creating the app is quick. Whether launch takes an hour or weeks depends on what comes after, the review and how long the provider takes. That's why guides show your own time separately from the provider's wait.

Three things came out of those conversations for me. A developer needs "how hard and how long" before they start, when they're deciding which apps to support, so each guide opens with that. Small copy-paste details cause a lot of the failures, since a wrong redirect address or a missing permission only shows up at the end, so the steps say exactly what to copy from where. And because the same questions come up for every tool, every guide uses one format and one research checklist, which is what would make a library of 100 apps comparable.

What I built

The home page is a search box and a library. If a guide exists it opens straight away. If not, the agent researches the app, streams its progress, and saves the result. Library cards show the easiest and the hardest way to set up each app, so the spread is visible before you click.



The Gmail guide on the live site, with the hardest setup selected.

A guide starts with "What are you building?" and everything below follows the setup you pick: its difficulty, its approval path (self-serve, app review or partner acceptance), your time, the provider's wait, and what the review checks. "Have ready" and "Watch out" list the requirements and the things that tend to go wrong, such as a customer's admin having to approve the app, limits before approval, or a user threshold before you may apply. "Stuck?" covers the common blockers. If no source gives a wait time, the guide says Unknown.

Every step is marked as yours, the agent's, or a wait on the provider. The agent's steps are the ones it can do through APIs, like creating the Composio auth config or drafting scope justifications. Logging in, accepting terms, recording a demo and clicking submit stay with the developer.

Each guide can be shared by link or copied as Markdown, and live generation works for apps outside Composio's catalog too.

How I worked

I started with the flows. I mapped who is involved (the developer, the provider, Composio, the end user) and where the developer gets stuck, and put the user flows and architecture on one page. Codex and a separate Claude agent reviewed that plan independently before I wrote code.

The first build had the site, the guide format, four guides researched in depth, a first version of the agent, and code that checks quotes. I also set up Slack end to end myself, so at least one guide was tested by doing. After that I worked on the categories (next section), then redesigned the guide page. The Gmail guide went from 5,350 visible words to 750, with the setup picker first and sources moved behind toggles. Then I rebuilt the agent around evidence, had Codex review it over six rounds until it judged it ready for live testing, and spent the rest of the time testing it on 18 apps.

A few decisions shaped most of the product:

- Knowledge comes from live research. A pre-built dataset is faster to query, but it can't answer for apps outside it and goes stale when docs change. The library is saved agent output.
- Difficulty is rated per setup, for the Gmail reason above.
- Time is split into your work and the provider's wait. "An hour of work and six weeks of review" needs a different plan from "a week of work".
- Claims are checked in code. A link from the model doesn't prove anything, so any claim whose quote fails the check is shown as unconfirmed.
- The agent works through APIs and a person does logins and terms. A bot clicking through developer consoles would break on two-factor logins. In the Slack test the agent did the setup and I clicked Allow once.

Sorting out the categories

To rate difficulty I needed categories, and my first attempt was a ladder of five levels, from self-serve up to a security assessment. I tested it against 24 providers, each read from its own public docs by a cheap AI reader. None of them fit cleanly.

So I looped: fold in what didn't fit, test 10 new providers, and repeat. After 7 rounds and 86 providers the categories stopped changing. The providers that still didn't fit were mostly ones whose docs leave things out.

Round	Version	Clean fits
1	Five-level ladder	0 of 24
2	Eight independent gates	0 of 12
3	Two axes	0 of 10
4	Added decision rules	2 of 10
5	Added use restrictions	4 of 10
6 and 7	Clarifications, stacked gates	3 and 4 of 10

What came out is three questions. What has to happen before a customer outside your company can connect: nothing, self-serve, an app review, or the provider accepting your company? If there's a review, what does it check: identity, use case and permissions, a working demo, security, or listing content? And what else changes the experience, such as a customer's admin approving the app, limits before approval, a usage threshold, paid access, prerequisites, use restrictions or repeat reviews?

A few things surprised me. The steps stack in any combination, so a ladder was the wrong shape from the start. An app review and a provider accepting your company are very different problems, because a review is a checklist you can prepare for and acceptance can simply be a no. 101 of Composio's 343 OAuth toolkits need no app at all, because their clients register themselves. And at providers like ServiceNow and Snowflake, each customer's admin sets up the integration in their own account.

How the agent works

The agent follows a fixed workflow. Code decides the order and the budgets, and models do the reading and writing. Each stage's model is a setting, so the same run could use Opus throughout without code changes.

It first identifies the provider from Composio's catalog and a short search, along with its official website and docs, and asks "did you mean" when a name is ambiguous. Only the provider's own domains count as documentation; look-alike domains and customer-hosted pages on shared platforms are excluded. It then finds pages from the docs' `llms.txt` index and four fixed searches, ranked so approval, review and limits pages come first and API reference pages last. Each page is read in full against a 13-question checklist, and every fact comes back with an exact quote. Up to three helpers then search for whatever the docs didn't answer, including developer reports of real wait times.

The writer can only use evidence it collected, and it cites evidence by ID. Code fills in each quote and link, so the writer has no way to invent a source. Then code checks every quote and number, and a judge model reads each claim, and each setup's path, wait and factors, against the quotes behind them. Anything unsupported is removed or marked unconfirmed. When the docs never mention a review for a setup, the guide calls self-serve "inferred" instead of stating it.

Testing it

To test the checker, I wrote 551 deliberately wrong versions of real claims, with wrong numbers, swapped units, and "must" and "may" flipped. Code on its own rejects or flags 56% of them, and the judge model reviews every claim on the important questions after that. Of 349 correct claims, none were wrongly marked unconfirmed. There are 124 automated tests in total.

For a hands-on check, the agent created a Slack app and the Composio auth config through their APIs, I clicked Allow once, and a real API call worked. That surfaced three things the docs don't make clear: Composio's API rejects a field name used in its own docs example, its Slack toolkit asks for user scopes rather than bot scopes, and connect links expire after 10 minutes.

The main test was 18 apps picked so that every path and factor shows up at least once: API-key-only apps, self-serve, customer admin approval, app review (always, for sensitive permissions, and above a usage threshold), partner acceptance, stacked gates, paid access and use restrictions. Each case lists the expected path, factors and must-have facts. These rounds ran on my Claude subscription so the API credit stayed for the live site.

Rounds	Apps fully right	What changed before them
1	9 of 18	First run of the rebuilt agent
2 to 4	12 to 14	Better page ranking, nested docs indexes, a pass that catches factors the writer left out
5 to 8	12 to 17	Docs on shared platforms (developer.zendesk.com) treated as official; clearer rules for the judge
9 and 10	18, then 17	The judge moved from Haiku to Sonnet

The biggest jump came from the judge rather than the research. With Haiku judging, the same agent scored 17 in one round and 12 in the next, because it kept rejecting correct answers over wording. Sonnet held at 18 and

17 for about three cents more per guide.

Some bugs only showed up on the real API. The reading model there spent its whole 3,000-token output limit thinking and returned no facts at all for Slack's most important pages. I raised the limit and made my test setup use the same limits as the API, so that can't hide again. Other runs read the wrong pages: one read six API reference pages for Gusto because their addresses contained "partner", and in another the writer's own note "no app review is mentioned" was taken as evidence that a review exists. In an earlier version, the agent claimed every Box enterprise admin must authorize your app. The page it read had lost its link text, so a rule for two app types read like a rule for all of them. Reading the docs' Markdown source fixed that.

Later re-checks with the API's exact limits passed 3 to 4 of 6 apps while Composio Search was returning empty results. Each miss came from pages that were never found, not from wrong facts.

Three of my own assumptions turned out wrong along the way. I thought Slack was fine without the Marketplace, but unlisted apps get one request a minute when reading channel history. I thought Shopify always needs a review, but custom apps for a single store don't. And I thought any restricted Gmail scope meant a security assessment, when it only applies if the data reaches your servers.

Where it stands

The library has 11 guides, 4 researched in depth and 7 written by the agent, and 357 of their 362 quotes are found word for word on the provider's page. A new guide costs about $0.15 to $0.25 on the API and takes around two minutes.

There are real limits. No guide has been reviewed by a person yet, and only Slack has been tested hands-on; every guide carries a badge saying so. The agent depends on search, so when Composio Search was slow it found fewer pages and wrote thinner guides. It now retries and reads in full any official pages its helpers find, but a search outage still costs quality. Providers rarely publish review times, so guides say Unknown more often than a developer would like. And the agent can't see anything behind a developer-console login.

What I'd do next

I'd get to 100 guides by running the agent over Composio's 219 bring-your-own toolkits, with a person reviewing them in order of how much each toolkit is used. I'd show the guide where developers actually get stuck, when they create an auth config or when a user hits "this app is blocked". I'd re-check every quote weekly against the live page (the script already exists) and re-run the agent when one breaks. The metric I'd watch is the time from a developer's first test connection to their first production connection on their own OAuth app.

What I took away

Setup was rarely the hard part. Distribution rules, domain checks and the wait on the provider were. I also came to prefer "Unknown" to a guess, because a guide that invents a wait time does more harm than one that admits it doesn't know. Testing my categories on real providers mattered more than designing them well on paper; my first model fit none of the first 24. On models, the cheapest one read pages fine, but the judge, which decides what the developer actually sees, needed the stronger one. And setting up Slack by hand found three things that reading the docs never would have.

How it's built

Next.js and TypeScript on Vercel, with guides stored in Postgres (Neon). Claude Haiku 5.5 identifies the app and reads pages; Claude Sonnet 5.5 writes the guide and judges it. Composio's toolkit API provides managed-app status and app details, and Composio Search finds pages. Guides follow a fixed schema (Zod), and live generation has a daily cap and a per-visitor hourly cap. I built it with Claude Code, had plans and code reviewed by Codex, and ran agent testing on my Claude subscription.

Terms used

OAuth app	Your registered identity with a provider such as Google: a client ID and secret, the permissions you ask for, and where users return after approving.
Client ID and secret	The OAuth app's credentials, copied from the provider's developer console into Composio.
Redirect address (redirect URI)	Where the provider sends a user after they approve. It has to match exactly, and some providers check that you own its domain.
Scopes	The permissions an app asks for. Sensitive ones usually mean a stricter review.
Managed app or your own	Composio lends its own approved app for many tools, which is fine for a prototype. Launch usually means your own app and the provider's approval process.
Auth config, connected account	In Composio, the auth config holds your app's credentials, and each end user who approves becomes a connected account.
App review, partner acceptance	In an app review the provider checks your app. With partner acceptance it must accept your company first, and it can say no.